

变物性扩散与移动边界下的 圆柱形药材烘干模型

摘 要

针对圆柱形中药材热风烘干中的温度滞后、低含水率扩散减速及失水收缩问题，建立径向非线性传热传质模型。以附件中的烘房环境作为对流边界输入，以药材干基含水率为状态量，并用实测半径构造随物质运动的收缩坐标。模型统一处理预热、恒温干燥及收缩过程，输出题目要求的空间与时间分辨率结果。

问题一采用给定常物性热参数及含水率相关扩散系数。1800 s 时，中心和表面温度分别为 33.5753℃、36.7856℃，对应干基含水率为 2.5500、1.5102 kg/kg，表明预热初期表层优先升温失水，中心基本保水。

问题二将密度、比热容、导热系数及扩散系数按附录三逐点更新，扩散公式中的温度严格换算为开尔文。3 h 时中心与表面温度分别为 49.8495℃、49.9664℃，含水率仍为 1.7662、1.0081 kg/kg，说明温度趋稳不等于干燥完成。

问题三以空间最大含水率首次下降至 0.15 kg/kg 为临界条件。在 4 h 后烘房保持 50℃、0.05 kg/kg 的假设下，临界时长为 57.4723 h；满足严格低于阈值的首个整秒为 206901 s，即 57.4725 h。问题四引入实测径向收缩并采用附录四物性，临界时长为 51.0906 h，首个达标整秒为 183927 s，即 51.0908 h。

数值上采用表面加密有限体积离散、扩散势积分通量及隐式变阶时间积分，避免低含水率边界层中的系数平均偏差。通过网格加密、圆柱热传导解析解对照、水分通量平衡与逐项文件核验检验计算结果。网格误差远小于环境与传质参数变化引起的时长差异。

关键词：药材烘干；非线性扩散；有限体积法；移动边界；干燥终点

1 问题分析、数据处理与基本假设

赛题要求预测长 25 cm、初始半径 2 cm 的圆柱药材在烘房中的温度与干基含水率分布，并判断固定尺寸与实测收缩条件下的干燥时间[1]。本研究只依据赛题经验式与附件数值建立模型，不引入未经标定的药材种类特异参数。

1.1 数据检查与延拓

附件一共有 241 个数值时点，采样间隔为 60 s，覆盖 0 ~ 14400 s；温度由 28℃升至约 50℃，烘房水分浓度由 0.01963 kg/kg 升至约 0.05 kg/kg。附件二共有 145 个时点，间隔为 1800 s，覆盖 0 ~ 72 h，半径由 2 cm 单调降至 1.198 cm[2]。时间序列无重复、缺失或逆序记录。两组观测均采用分段线性插值，避免高阶插值产生非物理振荡。

附件一最后一小时的温度与水分浓度均值分别为 49.9989℃、0.0499875 kg/kg，标准差为 0.1531℃、0.0001489 kg/kg。因此，0 ~ 4 h 保留全部观测波动，4 h 后分别保持 50℃及 0.05 kg/kg。这是后续时长预测必需的边界延拓假设，不是新增观测。若计算超出半径观测区间，则保持末次半径；本次问题四在 72 h 内结束，未使用半径外推。

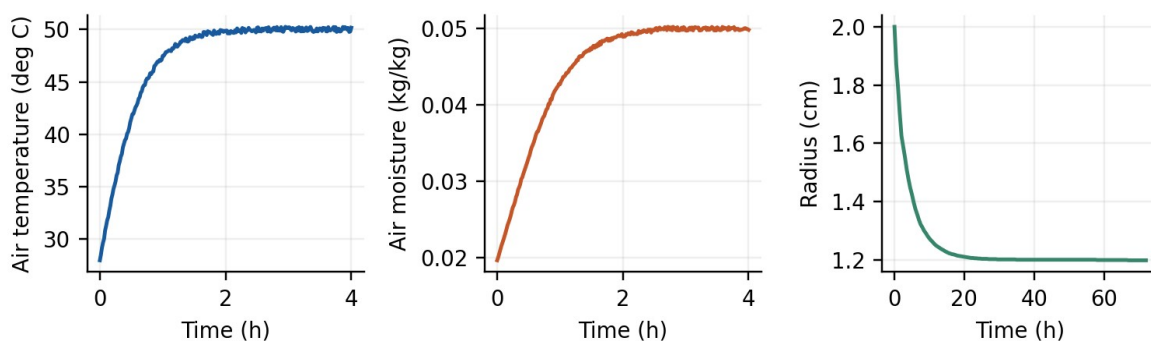


图 1 烘房边界输入及实测半径变化

1.2 建模假设与适用范围

(1) 药材截面轴对称，初始温度与含水率均匀；以长圆柱的中部横截面为研究对象，忽略轴向传热传质。两端面积与侧面积之比为 $R/L=0.08$ ，径向为主要传输方向；全域达标结论是在这一径向模型内作出的。

(2) 外界换热与传质系数在各问中保持 $h=25 \text{ W}/(\text{m}^2\cdot\text{K})$ 、 $h_m=8\times 10^{-7} \text{ m/s}$ 。将附件一水分浓度作为经验对流传质边界的有效平衡值；题目未给吸附等温线，因此不另行换算气相湿度与固体平衡含水率。

(3) 采用有效导热与有效扩散描述内部输运，忽略辐射、内部热源及未给定的蒸发潜热耦合；问题四假设径向均匀收缩，干物质随骨架运动且不流失。密度经验式仅用作热容系数，不据此反推收缩率或改变干基含水率定义。

2 固定圆柱的变物性传热传质模型

2.1 状态、守恒形式与初边值条件

符号	物理意义	单位
t, r	时间、距轴线径向距离	s, m
T, C	药材温度、干基含水率	°C, kg/kg
T_a, C_a	烘房温度、有效边界水分浓度	°C, kg/kg
R, x	药材半径、归一化物质坐标 r/R	m, 1
ρ, c_p, k	密度、比热容、导热系数	kg/m ³ , J/(kg·K), W/(m·K)
D	有效水分扩散系数	m ² /s

对半径 r 的环形控制体应用热传导和有效水分扩散平衡，有：

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rk(C)\frac{\partial T}{\partial r}\right)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rD(T, C)\frac{\partial C}{\partial r}\right), \quad 0 < r < R$$

初始条件为 $T(r,0)=28^\circ\text{C}$ 、 $C(r,0)=2.55 \text{ kg/kg}$ 。中心为对称零通量边界： $\partial T/\partial r = \partial C/\partial r = 0$ 。表面遵循第三类边界条件：

$$-k\frac{\partial T}{\partial r}\bigg|_R = h(T_s - T_a), \quad -D\frac{\partial C}{\partial r}\bigg|_R = h_m(C_s - C_a)$$

当药材温度低于烘房时，表面对外热通量为负，即净吸热；当含水率高于边界平衡值时，对外水分通量为正，即失水。 D 、 k 位于散度算子内部，不将非线性扩散误写成 $D\nabla^2 C$ 或将变导热系数移出散度。

2.2 各问物性参数

参数	问题一	问题二、三	问题四
ρ	820	650+128C	760+90C
c_p	2600	1450+2736C/(C+1)	1850+2150C/(C+1)
k	0.36	0.21+0.38C/(C+1)	0.12+0.20C/(C+1)
D	$7\times 10^{-9} \exp(-0.89/C)$	$2.4\times 10^{-3} \exp(-0.45/C-3850/T^k)$	$4.2\times 10^{-4} \exp(-0.30/C-3850/T^k)$

其中 $T^k=T+273.15$ 。问题一的热物性为常数，水分扩散仍为非线性；问题二与三从 $t=0$ 起统一采用附录三公式，不在 1800 s 处拼接问题一解。对流系数、初始状态与烘房输入保持一致。

3 收缩模型与数值求解

3.1 移动物质坐标

问题四取 $R(t)$ 为实测半径的线性插值。均匀径向收缩的骨架速度为 $u(r,t)=rR'(t)/R(t)$ ，令 $x=r/R(t)$ 。对随骨架运动的温度、干基含水率采用物质导数，坐标变换后得到：

$$\rho C_p \frac{\partial T}{\partial t} \Big|_x = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} \left(x k \frac{\partial T}{\partial x} \right), \quad \frac{\partial C}{\partial t} \Big|_x = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} \left(x D \frac{\partial C}{\partial x} \right)$$

表面边界相应为 $-kT_x/R=h(T_s-T_a)$ 、 $-DC_x/R=h_m(C_s-C_a)$ 。均匀收缩下，干物质体积密度按 R^{-2} 变化，该因子在干基含水率守恒式中约去，故不额外加入“压缩使 C 增大”的体积浓度项。移动坐标中的几何对流与骨架物质输运相抵消，不能只移动计算边界而把原药材当作静止介质裁切。

3.2 表面加密有限体积法与积分通量

在 $x \in [0,1]$ 上设 $N=800$ 个区间，节点与控制体权重定义如下。内部控制体界面取相邻节点中点，端点界面分别取 0、1。中心使用零面积边界通量，直接消除 $1/r$ 表观奇异性。

$$x_i = 1 - (1 - i/N)^{3/2}, \quad w_i = \frac{x_{i+1/2}^2 - x_{i-1/2}^2}{2}, \quad i = 0, \dots, N$$

将 D 写成 $b(T)\exp(-a/C)$ ，定义扩散势 Φ ，其导数为 $\exp(-a/C)$ ：

$$\Phi(C) = C \exp(-a/C) + a \operatorname{Ei}(-a/C), \quad \Phi'(C) = \exp(-a/C)$$

$$F_{i+1/2} = \frac{x_{i+1/2} b(T_{i+1/2})}{R^2} \frac{\Phi(C_{i+1}) - \Phi(C_i)}{x_{i+1} - x_i}, \quad \dot{C}_i = \frac{F_{i+1/2} - F_{i-1/2}}{w_i}$$

Ei 为指数积分函数。问题一 $b=7 \times 10^{-9}$ 、 $a=0.89$ ；问题二、三与四按相应附录取 b 、 a 。界面温度取算术均值；两端含水率非常接近时改用导数极限以避免相减消去。热通量使用导热系数的调和平均。表面水分通量取 $-h_m(C_s-C_a)/R$ ，中心通量为零。该构造显式保持离散通量平衡，且适合末期低扩散系数边界层。

3.3 时间积分、事件检测与结果采样

采用 SciPy 的 BDF 隐式变阶积分与稀疏雅可比结构[3]，相对容差 2×10^{-8} 、绝对容差 2×10^{-10} ；前 4 h 最大步长 20 s，此后最大步长 180 s。4 h 处拆段，状态连续传递。输出步长与内部积分步长分离：前两问以 1 s 采样，后两问以 60 s 采样，并补充终止时刻。半径方向将节点解线性插值到所需物理距离。

干燥临界事件为全部节点的最大含水率下降至 0.15 kg/kg，取下降方向根 t^* ；停机时间取严格大于 t^* 的首个整秒。计算使用未舍入数据判定，不能用显示为 0.1500 的结果单元格判断是否达标。

4 问题一：30 分钟预热平衡结果

表 1、表 2 分别列出题目要求时点的温度与水分浓度。列标题均为到药材中心的距离（cm），温度单位为℃，水分浓度单位为 kg/kg。

表 1 30 分钟内药材的温度（℃）

时间/s	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8488
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 2 30 分钟内药材的水分浓度（kg/kg）

时间/s	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

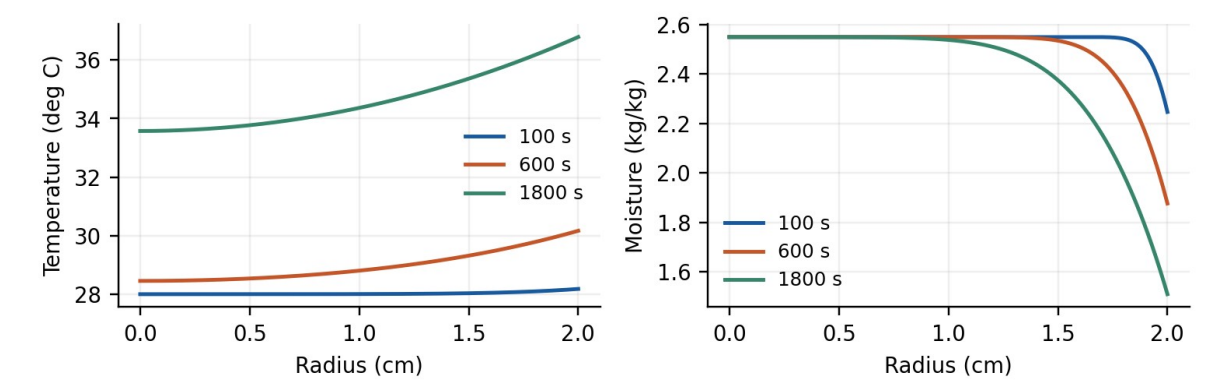


图 2 预热阶段的径向温度与水分浓度分布

热扩散率为 $k/(pc_p)=1.6886\times10^{-7} \text{ m}^2/\text{s}$ ，初始水分扩散系数约为 $4.9377\times10^{-9} \text{ m}^2/\text{s}$ ，传热快于水分迁移；但热边界并非瞬时跳变，药材中心仍明显滞后于烘房。表层水分优先向外排出，内部维持较高含水率，不能将表面状态代表整根药材。

result1.xlsx 含“温度”“水分浓度”两个工作表，各含 1~1800 s 的 1800 行数值；空间列为 0~2 cm、间隔 0.1 cm。初始条件在模型中明确给定，数据表按模板从 1 s 起列示。

5 问题二：全程变物性模型的前三小时结果

问题二与问题三共用附录三模型的同一条连续轨迹。表 3、表 4 列出其中前三小时，每隔 0.5 h 的径向分布；并非把问题一结果直接作为问题二的初值。

表 3 3 小时内药材的温度 (°C)

时间/h	0	0.5	1	1.5	2
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5137	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 3 小时内药材的水分浓度 (kg/kg)

时间/h	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

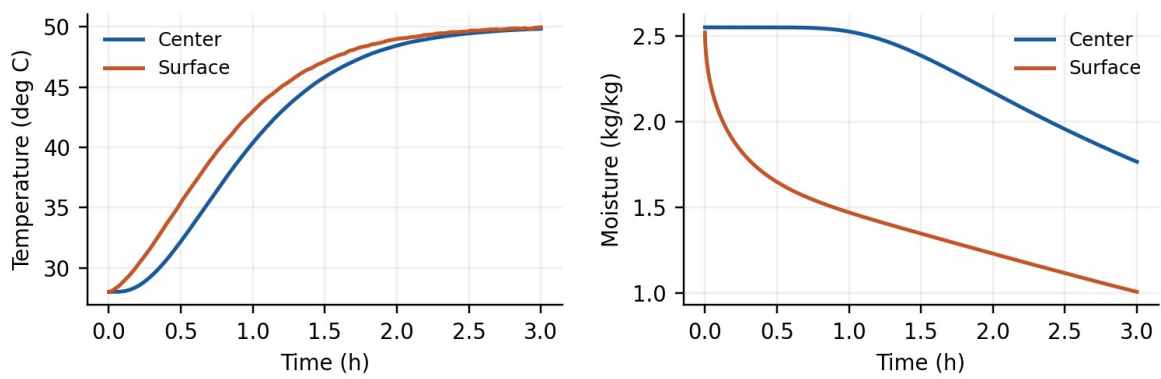


图 3 变物性模型前三小时的中心和表面状态

随温度升高，Arrhenius 因子推动扩散增强；随含水率降低， $\exp(-0.45/C)$ 又使扩散减弱，两种效应共同作用。3 h 时温度已接近烘房水平，水分浓度仍远高于 0.15 kg/kg。因而仅由升温曲线或热平衡时间确定停机，将严重低估所需干燥时间。

result2.xlsx 对应本问规定的 0 ~ 3 h 结果区间，两个工作表分别保存 1 ~ 10800 s 的温度、水分浓度，径向间隔 0.1 cm；同一全程模型继续积分至干燥结束，其全程分钟级水分结果见 result3.xlsx。

6 问题三：固定半径的干燥时长

未舍入的最大含水率首次达到阈值的时刻为 $t^*=206900.319008\text{ s}$ ，即 57.4723 h 。严格满足各处 $C<0.15\text{ kg/kg}$ 的首个整秒为 206901 s ，即 57.4725 h ；若只允许整分钟停机，可取 3449 min （ 57.4833 h ）。

表 5 药材烘干过程的水分浓度（kg/kg）

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
6.0000	1.0170	0.9881	0.9015	0.7550	0.5328
12.0000	0.4561	0.4432	0.4031	0.3297	0.1642
18.0000	0.2988	0.2915	0.2687	0.2253	0.0845
24.0000	0.2378	0.2327	0.2167	0.1855	0.0665
30.0000	0.2058	0.2018	0.1892	0.1641	0.0598
36.0000	0.1859	0.1825	0.1718	0.1504	0.0566
42.0000	0.1721	0.1691	0.1597	0.1407	0.0549
48.0000	0.1618	0.1592	0.1507	0.1334	0.0537
54.0000	0.1539	0.1514	0.1436	0.1277	0.0530
57.4725	0.1500	0.1477	0.1402	0.1249	0.0527

表中最后一行为烘干结束时刻。中心至表面的浓度在全过程保持不增，中心控制终点；判断时仍对全部计算节点取最大值，不预先把事件函数简化为某一个观测点。

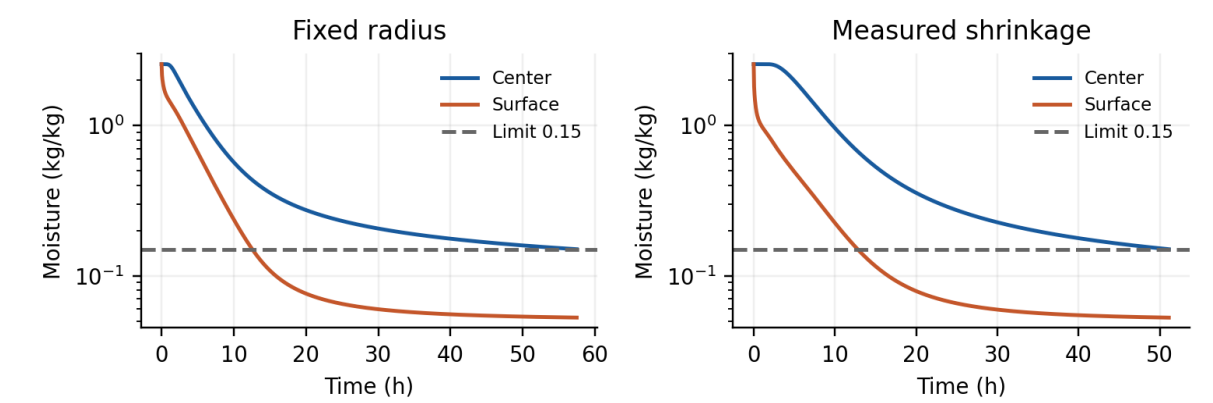


图 4 固定半径与实测收缩条件下的中心、表面水分轨迹

图 4 使用对数纵轴显示末期减速。表层虽先接近边界平衡含水率，但内部水分继续向外迁移。低含水率时有效扩散系数指数下降，最后一段干燥占据较长时间。停止条件是空间最大值，而不是截面平均值，也不是表面含水率。

result3.xlsx 按模板从 60 s 起每隔 60 s 保存，共 3449 行数据，末行补入 206901 s；各空间位置均以实际的 0 ~ 2 cm 坐标表示。

7 问题四：实测收缩下的干燥时长

在附录四物性与实测收缩半径共同作用下，临界时刻为 183926.097408 s，即 51.0906 h；首个达标整秒为 183927 s，即 51.0908 h。相应表面位置为 1.2000 cm，全部取样点必须位于该时刻药材内部。

表 6 收缩药材的水分浓度 (kg/kg) 及实际表面位置

时间/h	0 cm	0.5 cm	1 cm	药材表面	表面位置/cm
6.0000	1.7196	1.5377	1.0226	0.4207	1.3740
12.0000	0.7377	0.6546	0.4083	0.1670	1.2480
18.0000	0.4087	0.3686	0.2397	0.0892	1.2140
24.0000	0.2851	0.2611	0.1790	0.0673	1.2040
30.0000	0.2263	0.2094	0.1493	0.0595	1.2010
36.0000	0.1928	0.1797	0.1317	0.0560	1.2000
42.0000	0.1712	0.1604	0.1200	0.0541	1.2000
48.0000	0.1561	0.1468	0.1116	0.0530	1.2000
51.0908	0.1500	0.1413	0.1081	0.0526	1.2000

题目表 6 的最早时点为 6 h，此时半径已小于 1.5 cm，故正文表中只列有效的 0、0.5、1 cm 固定坐标，并另列真实表面；不将固定的 1 cm 列误标为表面。

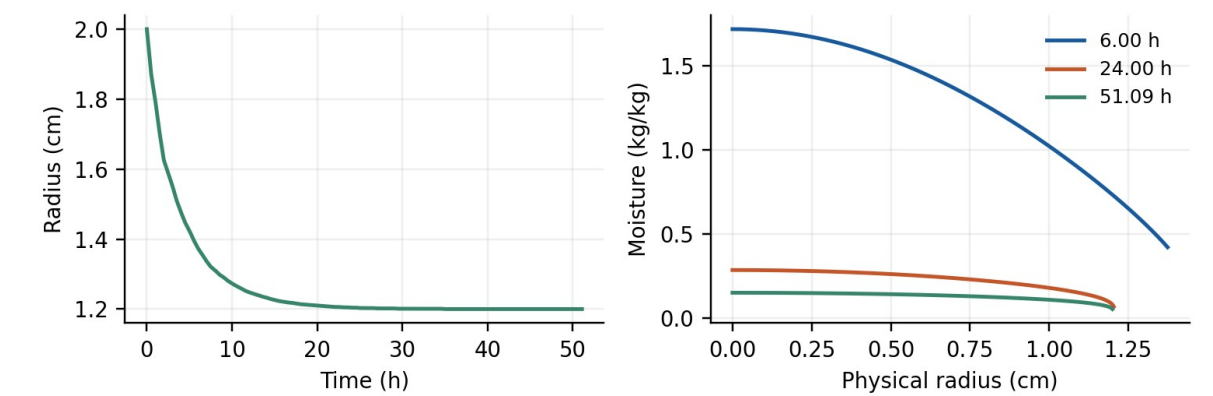


图 5 收缩半径及不同时间的真实空间水分剖面

result4.xlsx 的 Sheet1 以时间为 A 列，其后依次为 0~1.9 cm、间隔 0.1 cm 的固定物理坐标，最后一列为“药材表面”。某固定坐标大于当时 $R(t)$ 时留空，表示该点已位于药材外部；不是零浓度或缺失求解。表面值总是由 $x=1$ 处求得。“表面位置”工作表给出同步半径，用于还原每行的空间范围。

文件从 60 s 起每隔 60 s 保存，末行补入 183927 s，共 3066 行。结果比问题三提前 6.3817 h，但两问同时改变了物性公式与尺寸，不能将这一净差值全部归因于收缩；下一节用同物性对照分离几何影响。

8 数值验证与误差分析

8.1 空间网格加密

采用相同的加密映射和时间积分容差，分别计算 N=400、800、1600。表 7 的浓度差为与交付网格 N=800 相比，在 1、2、3、10、30 s 及之后 200 个时点、0~2 cm 每隔 0.5 cm 的有效物理位置上的最大绝对差；问题四排除已移到域外的点。

表 7 网格加密与主网格结果的差异

模型	N	临界时间/h	最大温差/°C	最大浓度差
问题一	400	—	1.07e-05	3.49e-05
问题二、三	400	57.472343	1.54e-05	3.00e-05
问题四	400	51.090613	8.53e-06	1.76e-05
问题一	1600	—	1.16e-05	8.69e-06
问题二、三	1600	57.472303	5.61e-06	7.40e-06
问题四	1600	51.090574	3.09e-06	2.36e-06

N=800 与 1600 的临界时间差均小于 0.04 s，远小于结果表的 60 s 长程采样间隔。空间误差按加密继续下降；温度差受自适应积分及输入折线拐点误差共同影响，不据其非单调变化夸大收敛阶。四位小数是统一输出精度，不代表经验参数具有同等实验准确度。

8.2 独立热传导解析解对照

保留问题一常热物性，将烘房温度改为恒定 50°C，用圆柱第三类边界的 Bessel 级数解对照同一离散求解器：

$$\frac{T-T_a}{T_0-T_a} = \sum_{n=1}^{\infty} A_n J_0(\lambda_n r/R) \exp(-\alpha \lambda_n^2 t/R^2)$$

$$\lambda_n J_1(\lambda_n) = Bi J_0(\lambda_n), \quad A_n = \frac{2J_1(\lambda_n)}{\lambda_n [J_0(\lambda_n)^2 + J_1(\lambda_n)^2]}$$

其中 $Bi=hR/k$ ， $\alpha=k/(\rho c_p)$ 。特征值由区间括根求出，保留 $\lambda \leq 160$ 的所有根，在 100、600、1800 s 和五个指定半径处比较。

最大温度绝对误差为 5.21e-06°C。该检验独立于附件温度拟合，能够检查中心对称边界、表面对流符号及圆柱几何系数。

8.3 水分平衡、极值与文件核验

$$\bar{C} = 2 \sum_i w_i C_i, \quad \frac{d\bar{C}}{dt} = -\frac{2h_m}{R} (C_s - C_a)$$

这一平衡在固定半径下为截面平均水分收支，在均匀收缩假设下对应物质坐标中的干物质加权平均。三组计算的最大瞬时离散残差均小于 $6 \times 10^{-20} \text{ s}^{-1}$ 。检查全部输出轨迹无负浓度，径向浓度增加量仅为浮点舍入量级；所有结果表逐单元格对照未舍入 CSV，数值误差均不超过四位小数舍入界限 0.00005。

9 边界敏感性、收缩作用与模型评价

9.1 恒温条件与传质、扩散系数敏感性

保持其余因素不变，用 $N=400$ 加密网格逐项扰动。 h_m 扰动作用于全程， D 扰动为扩散式整体乘数，温度扰动仅作用于 4 h 以后。表 8 给出条件变化下的临界时长，作为模型场景对比，不作为统计置信区间。

表 8 参数扰动后的临界烘干时长 (h)

情景	固定半径 (附录三)	实测收缩 (附录四)
基准 ($N=800$)	57.4723	51.0906
末次观测值恒定延拓	57.1694	50.8242
恒温阶段 49.5℃	58.3955	51.9002
恒温阶段 50.5℃	56.5696	50.2987
传质系数 -10%	58.2082	51.3818
传质系数 +10%	56.8950	50.8697
扩散系数 -10%	63.1303	56.0903
扩散系数 +10%	52.8651	47.0111

末次观测值为 50.165℃、0.04986 kg/kg，其恒定延拓与“末段均值归整为稳态设定值”并不完全相同，因此会改变长程终点。数据并未给出 4 h 后的实际控制记录，绝对时长应连同本文边界延拓假设解释。

9.2 同物性对照下的纯几何收缩效应

保持附录四所有经验式不变，仅把实测 $R(t)$ 改成固定 2 cm，预测临界时长为 129.8442 h；采用实测收缩时为 51.0906 h (两者均 $N=400$)。同物性下，收缩使时长缩短 78.7536 h，约 60.65%。这组对照比直接相减问题三、四更能识别尺寸变化的作用。

收缩同时缩短内部扩散路径并增大单位物质坐标体积的表面传质系数；归一化方程中分别体现为 $1/R^2$ 与 $1/R$ 。最终半径约为初始值的 60%，径向扩散尺度 R^2 约降至初始值的 36%。但 $R(t)$ 随时间变化，且 D 对水分呈指数依赖，因此不能简单把全程时间乘以 0.36。

9.3 优点、局限与结论

模型直接保留非线性通量、实测边界和收缩坐标，能够给出全部时空结果及可重复的终点判定。其主要局限来自径向一维化、等效气固水分边界、均匀骨架收缩和忽略潜热；目前附件没有内部温湿度测量，故本文完成的是数值一致性与情景敏感性验证，而非对真实药材的实验验证。若进一步用于工艺设计，应测定平衡吸附关系与潜热耦合，并用中心测温和分层含水率观测校准。

在本文约定的边界与物理假设下，问题三、四建议整秒停机时长分别为 57.4725 h、51.0908 h；四份结果文件与正文各表均来自同一组未舍入求解轨迹。

参考文献

[1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 A 题：药材的烘干问题[Z]. 竞赛赛题文件，2026.

[2] 全国大学生数学建模竞赛 A 题附件. 附件 1.xlsx；附件 2.xlsx；附件 3 结果模板[DS]. 赛题配套数据，2026.

[3] SciPy Developers. scipy.integrate.solve_ivp[EB/OL]. SciPy 官方文档，访问日期：2026-09-10. https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html.

附录 A 支撑材料清单及复现说明

支撑材料压缩包保留以下文件，全部为匿名电子材料。NPZ 密集节点中间轨迹由 solve.py 在复现时生成，未重复放入压缩包；相应图和题目要求的输出已完整保存。

相对路径	内容与用途
result1.xlsx ~ result4.xlsx	四份指定结果文件
data/air.csv，data/radius.csv	原附件数值的无损 CSV 转录
code/solve.py	全部模型、网格、积分、事件、收敛及敏感性分析
code/plot_results.py	五幅论文图的生成程序
code/verify.py	工作簿数值核验及解析解对照
code/export_results.mjs	结果表格式化导出程序
results/result*_T.csv，result*_C.csv	未按四位小数舍入的时空结果对照文件
results/radius_output.csv	问题四每一输出时刻的表面位置
results/summary.json，verification.json	正文表格、时长、收敛、敏感性及核验结果
figures/01_inputs.png ~ 05_shrinkage.png	论文使用的五幅图
README.md，requirements.txt	执行步骤、依赖版本与字段说明

标准 Python 环境安装 requirements.txt 中列出的依赖后，依次运行 python code/solve.py --n 800 --validation、python code/plot_results.py、python code/verify.py。核心建模程序不依赖电子表格软件或专用平台，可直接重新生成所有数值 CSV、时长和验证数据；verify.py 对照压缩包中的四份工作簿。

export_results.mjs 仅用于已有数值的工作簿排版，依赖 @oai/artifact-tool（2.8.58 或以上）的 Node 环境，不参与模型计算。可在具有该依赖的运行环境中逐次以参数 1、2、3、4 执行。无此依赖时，核心 Python 复现与数值核验仍可完整运行，CSV 与已交付工作簿一一对应。

AI 辅助说明：本文建模实现、数值分析和文稿整理使用 AI 工具协助完成；论文中的数值均由随附程序实际计算产生，未将生成文本当作实验观测。

附录 B 完整数值求解源程序

文件: code/solve.py

```
"""Axisymmetric drying model. Run from any directory; outputs ../results.
Python 3.12, numpy, scipy, openpyxl (read only), matplotlib.
No random numbers are used. Temperatures in deg C except inside Arrhenius law.
"""
from pathlib import Path
import argparse, json, time
import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import diags, bmat
from scipy.special import expi

ROOT = Path(__file__).resolve().parents[1]
DATA, OUT = ROOT / 'data', ROOT / 'results'

class Drying:
    def __init__(self, mode, n=800, tail=(50., .05), hm=8e-7,
                 h=25., diffusion_scale=1., shrink=None):
        self.mode, self.n = mode, n
        self.tail, self.hm, self.h = tail, hm, h
        self.scale = diffusion_scale
        self.shrink = mode == 4 if shrink is None else shrink
        self.air = np.loadtxt(DATA / 'air.csv', delimiter=',', skiprows=1)
        self.rad = np.loadtxt(DATA / 'radius.csv', delimiter=',', skiprows=1)
        self.x = 1-(1-np.linspace(0., 1., n+1))*1.5
        self.dx = np.diff(self.x)
        self.faces = np.r_[0., (self.x[:-1]+self.x[1:])/2, 1.]
        self.vol = np.diff(self.faces**2)/2
        a = diags([np.ones(n), np.ones(n+1), np.ones(n)], [-1,0,1])
        self.pattern = bmat([[a,a],[a,a]], format='csc')

    def radius(self, t):
        return np.interp(t, self.rad[:,0], self.rad[:,1])/100 if self.shrink else .02

    def ambient(self, t):
        if t > self.air[-1,0]:
            return self.tail
        return tuple(np.interp(t,self.air[:,0],self.air[:,j]) for j in (1,2))

    def properties(self, T, C):
        c = np.maximum(C, 1e-10)
        if self.mode == 1:
            rho, cp, k = 820., 2600., np.full_like(c,.36)
            D = 7e-9*np.exp(-.89/c)
        elif self.mode == 2:
            rho, cp, k = 650+128*c, 1450+2736*c/(1+c), .21+.38*c/(1+c)
            D = 2.4e-3*np.exp(-.45/c-3850/(T+273.15))
        else:
            rho, cp, k = 760+90*c, 1850+2150*c/(1+c), .12+.20*c/(1+c)
            D = 4.2e-4*np.exp(-.30/c-3850/(T+273.15))
        return rho*cp, k, D*self.scale

    def diffusion(self, z, coeff, transfer, ambient, R):
        face_coeff = 2*coeff[:-1]*coeff[1:]/(coeff[:-1]+coeff[1:])
        flux = np.empty(self.n+2)
        flux[0] = 0.
        flux[1:-1] = self.faces[1:-1]*face_coeff*np.diff(z)/self.dx/R**2
        flux[-1] = -transfer*(z[-1]-ambient)/R
        return np.diff(flux)/self.vol

    def rhs(self, t, y):
        m = self.n+1
        T, C = y[:m], y[m:]
        Ta, Ca = self.ambient(t)
        R = self.radius(t)
        capacity,k,D = self.properties(T,C)
        # Kirchhoff integrated moisture flux avoids harmonic-mean bias in the
        # very dry surface layer. Integral exp(-a/C)dC = C exp(-a/C)+a Ei(-a/C).
        a = {1:.89,2:.45,4:.30}[self.mode]
        c = np.maximum(C,1e-10)
        potential = c*np.exp(-a/c)+a*expi(-a/c)
        delta = np.diff(C)
        q = np.diff(potential)
```

```

close = abs(delta)<1e-6*np.maximum(c[:-1],c[1:])
q[close] = np.exp(-a/((c[:-1]+c[1:])/2))[close]*delta[close]
if self.mode==1: prefactor=7e-9
else:
    prefactor=(2:2.4e-3,4:4.2e-4)[self.mode]
    *np.exp(-3850/((T[:-1]+T[1:])/2+273.15)))
flux = np.r_[0.,self.faces[1:-1]*prefactor*self.scale*q/self.dx/R**2,
             -self.hm*(C[-1]-Ca)/R]
return np.r_[self.diffusion(T,k,self.h,Ta,R)/capacity,
            np.diff(flux)/self.vol]

def run(self, until=None, rtol=2e-8, atol=2e-10):
    end = until if until is not None else 14*86400.
    y = np.r_[np.full(self.n+1,28.),np.full(self.n+1,2.55)]
    def event(t,z): return np.max(z[self.n+1:])-0.15
    event.direction, event.terminal = -1, until is None
    self.parts = []
    for a,b,step in [(0.,min(end,14400.),20.), (14400.,end,180.)]:
        if b <= a: continue
        sol = solve_ivp(self.rhs,(a,b),y,method='BDF',rtol=rtol,
                        atol=atol,max_step=step,dense_output=True,
                        jac_sparsity=self.pattern,events=event)
        if not sol.success: raise RuntimeError(sol.message)
        self.parts.append(sol)
        if len(sol.t_events[0]) and until is None:
            self.crossing = float(sol.t_events[0][0])
            break
        y = sol.y[:,-1]
    if until is None:
        roots = [t for p in self.parts for t in p.t_events[0]]
        if not roots: raise RuntimeError('No drying threshold reached')
        self.crossing = float(roots[0])
        self.stop = int(np.floor(self.crossing))+1
        extra = solve_ivp(self.rhs,(self.crossing,self.stop),self.parts[-1].y[:,-1],
                          method='BDF',rtol=rtol,atol=atol,max_step=1.,
                          jac_sparsity=self.pattern,dense_output=True)
        if not extra.success: raise RuntimeError(extra.message)
        self.parts.append(extra)
    else:
        self.crossing, self.stop = None, float(until)
    return self

def evaluate(self,t):
    t = np.atleast_1d(t).astype(float)
    y = np.empty((2*(self.n+1),len(t)))
    for j,p in enumerate(self.parts):
        mask = (t>=p.t[0]) & (t<=p.t[-1])
        if np.any(mask): y[:,mask] = p.sol(t[mask])
    return y

def sample(self,t,distances_cm):
    t = np.atleast_1d(t).astype(float)
    y = self.evaluate(t)
    m=self.n+1
    T,C = [],[]
    for j,s in enumerate(t):
        q=np.asarray(distances_cm)/100/self.radius(s)
        T.append(np.interp(q,self.x,y[:m,j],right=np.nan))
        C.append(np.interp(q,self.x,y[m:,j],right=np.nan))
    return np.array(T),np.array(C)

def write_csv(path,t,a,header):
    np.savetxt(path,np.column_stack([t,a]),delimiter=',',
               header=header,comments='',fmt='%.10g')

def export_case(model,label,dt,end=None):
    end=model.stop if end is None else end
    ts=np.arange(dt,end+1e-9,dt,dt=float)
    if ts.size==0 or ts[-1]<end: ts=np.r_[ts,end]
    distances=np.arange(21)/10
    T,C=model.sample(ts,distances)
    header='time_s,'+',','.join(f'{r:g}' for r in distances)
    if label in (1,2): write_csv(OUT/f'result{label}_T.csv',ts,T,header)
    if label==4:
        surface=model.evaluate(ts)[-1]
        C=np.column_stack([C[:20],surface])
        header='time_s,'+',','.join(f'{r:g}' for r in distances[:20])+',surface'

```

```

        write_csv(OUT/'radius_output.csv',ts,np.array([model.radius(t)*100 for t in
ts]),'time_s,radius_cm')
        write_csv(OUT/f'result{label}_C.csv',ts,C,header)
        return ts,T,C

def audit(model):
    ts=np.linspace(0,model.stop,1001)
    y=model.evaluate(ts); m=model.n+1
    C=y[m:]
    residual=[]
    for t,z in zip(ts[::10],y.T[::10]):
        dc=model.rhs(t,z)[m:]
        residual.append(abs(2*np.dot(model.vol,dc)+2*model.hm/model.radius(t)*(z[-1]-model.ambient(t)[1])))
    return dict(min_C=float(C.min()),max_C=float(C.max()),
        radial_increase_max=float(np.maximum(np.diff(C,axis=0),0).max()),
        balance_residual_max=float(max(residual)),
        stop_s=model.stop,crossing_s=model.crossing,
        stop_max_C=float(model.evaluate([model.stop])[m:].max()),
        prev_second_max_C=float(model.evaluate([model.stop-1])[m:].max()),
        nfev=sum(p.nfev for p in model.parts))

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=800)
    ap.add_argument('--validation',action='store_true');args=ap.parse_args()
    OUT.mkdir(parents=True,exist_ok=True)
    summary={'n':args.n,'tail':[50.,.05],'rtol':2e-8,'atol':2e-10}
    cases={}
    for mode,label,end in [(1,'q1',1800),(2,'q23',None),(4,'q4',None)]:
        start=time.time();m=Drying(mode,args.n).run(end);cases[label]=m
        summary[label]=audit(m)
        print(label,summary[label], 'elapsed',time.time()-start,flush=True)
    export_case(cases['q1'],1,1)
    export_case(cases['q23'],2,1,10800)
    export_case(cases['q23'],3,60)
    export_case(cases['q4'],4,60)
    # These coarse profiles are unrounded sources for figures and paper tables.
    for label,m in cases.items():
        times=np.linspace(0,m.stop,1001)
        vals=m.evaluate(times)
        np.savez_compressed(OUT/f'{label}_profiles.npz',times=times,x=m.x,
            T=vals[:m.n+1].T,C=vals[m.n+1:].T,
            R=np.array([m.radius(t) for t in times]))
    for key,times in [(('q1',[100,300,600,900,1200,1500,1800]),
        ('q23',np.arange(1800,10801,1800)))]:
        T,C=cases[key].sample(times,[0,.5,1,1.5,2])
        summary[key]['paper_t']=list(map(float,times))
        summary[key]['paper_T']=T.tolist();summary[key]['paper_C']=C.tolist()
    for key in ['q23','q4']:
        m=cases[key];ts=np.r_[np.arange(21600,m.stop,21600),m.stop]
        _,C=m.sample(ts,[0,.5,1,1.5,2])
        summary[key]['long_t']=ts.tolist()
        summary[key]['long_C']=C.tolist()
        summary[key]['long_surface']=m.evaluate(ts)[-1].tolist()
        summary[key]['long_radius']=m.radius(t)*100 for t in ts]
    if args.validation:
        summary['convergence']=[]
        for n in [args.n//2,args.n*2]:
            for mode,key,end in [(1,'q1',1800),(2,'q23',None),(4,'q4',None)]:
                m=Drying(mode,n).run(end)
                ref=cases[key]
                ts=np.unique(np.r_[1,2,3,10,30,np.linspace(60,min(m.stop,ref.stop),200)])
                Ta,Ca=m.sample(ts,[0,.5,1,1.5,2]);Tb,Cb=ref.sample(ts,[0,.5,1,1.5,2])
                row=dict(n=n,case=key,crossing_s=m.crossing,
                    max_T_diff=float(np.nanmax(abs(Ta-Tb))),
                    max_C_diff=float(np.nanmax(abs(Ca-Cb))))
                summary['convergence'].append(row);print('convergence',row,flush=True)
        summary['sensitivity']=[]
        tests=[('tail_last',dict(tail=tuple(cases['q23'].air[-1,1:])),
            ('tail_lowT',dict(tail=(49.5,.05))),('tail_highT',dict(tail=(50.5,.05))),
            ('hm_-10%',dict(hm=7.2e-7)),('hm_+10%',dict(hm=8.8e-7)),
            ('D_-10%',dict(diffusion_scale=.9)),('D_+10%',dict(diffusion_scale=1.1))]
        for mode,key in [(2,'q23'),(4,'q4')]:
            for name,opts in tests:
                m=Drying(mode,args.n//2,**opts).run()
                row=dict(case=key,test=name,hours=m.crossing/3600)
                summary['sensitivity'].append(row);print('sensitivity',row,flush=True)
        for shrink in [False,True]:

```

```

        m=Drying(4,args.n//2,shrink=shrink).run()
        summary['sensitivity'].append(dict(case='q4',test='shrink_'+str(shrink),hours=m.crossing/3600))
# JSON null / CSV nan mean physical points outside the shrunken cylinder.
def clean(obj):
    if isinstance(obj,dict): return {k:clean(v) for k,v in obj.items()}
    if isinstance(obj,list): return [clean(v) for v in obj]
    if isinstance(obj,float) and not np.isfinite(obj): return None
    return obj
(OUT/'summary.json').write_text(json.dumps(clean(summary),ensure_ascii=False,indent=2,allow_nan=False))

if __name__=='__main__': main()

```

附录 C 绘图源程序

文件: code/plot_results.py

```
"""Regenerate all paper figures from solve.py outputs; no GUI required."""
from pathlib import Path
import json
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT=Path(__file__).resolve().parents[1]
OUT=ROOT/'figures';OUT.mkdir(exist_ok=True)
S=json.loads((ROOT/'results/summary.json').read_text())
P={k:np.load(ROOT/f'results/{k}_profiles.npz') for k in ['q1','q23','q4']}
plt.rcParams.update({'font.family':'DejaVu Sans','font.size':9,
                    'axes.spines.top':False,'axes.spines.right':False,
                    'figure.dpi':120,'savefig.dpi':220,'axes.grid':True,
                    'grid.alpha':.2,'lines.linewidth':1.6})
colors=['#185A9D','#C4562A','#37866B','#895D9A']
def finish(fig,name):
    fig.tight_layout(pad=1.2);fig.savefig(OUT/name,bbox_inches='tight');plt.close(fig)

a=np.loadtxt(ROOT/'data/air.csv',delimiter=',',skiprows=1)
r=np.loadtxt(ROOT/'data/radius.csv',delimiter=',',skiprows=1)
fig,axs=plt.subplots(1,3,figsize=(7,2.2))
for ax,j,label in [(axs[0],1,'Air temperature (deg C)'),(axs[1],2,'Air moisture (kg/kg)')]:
    ax.plot(a[:,0]/3600,a[:,j],color=colors[j-1]);ax.set(xlabel='Time (h)',ylabel=label)
axs[2].plot(r[:,0]/3600,r[:,1],color=colors[2]);axs[2].set(xlabel='Time (h)',ylabel='Radius (cm)')
finish(fig,'01_inputs.png')

p=P['q1'];fig,axs=plt.subplots(1,2,figsize=(7,2.4))
for j,t in enumerate([100,600,1800]):
    for ax,field in zip(axs,['T','C']):
        profile=np.array([np.interp(t,p['times'],col) for col in p[field].T])
        ax.plot(p['x']*2,profile,label=f'{t} s',color=colors[j])
    for ax,y in zip(axs,['Temperature (deg C)','Moisture (kg/kg)']):
        ax.set(xlabel='Radius (cm)',ylabel=y);ax.legend(frameon=False,fontsize=8)
finish(fig,'02_preheat.png')

p=P['q23'];fig,axs=plt.subplots(1,2,figsize=(7,2.4))
for ax,field in zip(axs,['T','C']):
    a=np.loadtxt(ROOT/f'results/result2_{field}.csv',delimiter=',',skiprows=1)
    ax.plot(a[:,0]/3600,a[:,1],label='Center',color=colors[0])
    ax.plot(a[:,0]/3600,a[:,1],label='Surface',color=colors[1]);ax.set_xlabel('Time (h)')
    ax.legend(frameon=False,fontsize=8)
axs[0].set_ylabel('Temperature (deg C)');axs[1].set_ylabel('Moisture (kg/kg)')
finish(fig,'03_first3h.png')

fig,axs=plt.subplots(1,2,figsize=(7,2.5))
for ax,key,title in zip(axs,['q23','q4'],['Fixed radius','Measured shrinkage']):
    p=P[key];t=p['times']/3600
    ax.plot(t,p['C'][:,0],label='Center',color=colors[0])
    ax.plot(t,p['C'][:,1],label='Surface',color=colors[1])
    ax.axhline(.15,color='#666666',linestyle='--',label='Limit 0.15')
    ax.set(xlabel='Time (h)',ylabel='Moisture (kg/kg)',title=title,yscale='log',ylim=(.045,3))
    ax.legend(frameon=False,fontsize=7.5)
finish(fig,'04_drying.png')

p=P['q4'];fig,axs=plt.subplots(1,2,figsize=(7,2.5))
axs[0].plot(p['times']/3600,p['R']*100,color=colors[2])
axs[0].set(xlabel='Time (h)',ylabel='Radius (cm)')
for j,t in enumerate([6*3600,24*3600,S['q4']['stop_s']]):
    profile=np.array([np.interp(t,p['times'],col) for col in p['C'].T])
    radius=np.interp(t,p['times'],p['R'])
    axs[1].plot(p['x']*radius*100,profile,color=colors[j],label=f'{t/3600:.2f} h')
axs[1].set(xlabel='Physical radius (cm)',ylabel='Moisture (kg/kg)')
axs[1].legend(frameon=False,fontsize=8)
finish(fig,'05_shrinkage.png')
print('Saved five figures to',OUT)
```

附录 D 核验源程序

文件：code/verify.py

```
"""Independent saved-file checks plus an analytic constant-property heat test."""
from pathlib import Path
import json
import numpy as np
import openpyxl
from scipy.special import j0,j1
from scipy.optimize import brentq
from solve import Drying

ROOT=Path(__file__).resolve().parents[1]
def run():
    result={}
    for label in [1,2,3,4]:
        wb=openpyxl.load_workbook(ROOT/f'result{label}.xlsx',data_only=True,read_only=True)
        fields=[('温度','T'),('水分浓度','C')] if label<=2 else [('Sheet1','C')]
        for sheet,field in fields:
            expected=np.loadtxt(ROOT/f'results/result{label}_{field}.csv',delimiter=',',skiprows=1)
            rows=list(wb[sheet].values)
            saved=np.array([[np.nan if v is None else v for v in row] for row in rows[1:]],float)
            assert saved.shape==expected.shape,(label,sheet,saved.shape,expected.shape)
            assert np.array_equal(np.isnan(saved),np.isnan(expected))
            assert np.max(abs(saved[:,0]-expected[:,0]))==0
            err=float(np.nanmax(abs(saved[:,1:]-expected[:,1:])))
            assert err<=.00005001,err
            result[f'result{label}_{field}']={'rows':len(saved),'columns':saved.shape[1],
                                             'rounding_error_max':err,'last_time_s':saved[-1,0]}

    # For constant ambient 50 deg C, a cylinder has a Bessel-series solution.
    m=Drying(1,800);m.air=np.array([[0.,50.,.05],[1800.,50.,.05]])
    m.run(1800.,rtol=2e-10,atol=2e-12)
    Bi=25*.02/.36
    def equation(z): return z*j1(z)-Bi*j0(z)
    grid=np.linspace(1e-8,160,20000);roots=[]
    for a,b in zip(grid[:-1],grid[1:]):
        if equation(a)*equation(b)<0: roots.append(brentq(equation,a,b))
    roots=np.array(roots)
    coeff=2*j1(roots)/(roots*(j0(roots)**2+j1(roots)**2))
    times=np.array([100.,600.,1800.]);dist=np.array([0.,.5,1,1.5,2])
    numerical=m.sample(times,dist)[0]
    analytic=np.array([50+(28-50)*np.sum(coeff[:,None]*j0(roots[:,None]*dist[None,:])/2)
                      *np.exp(-.36/(820*2600)*roots[:,None]**2*t/.02**2),axis=0)
                      for t in times])
    result['analytic_heat_max_error_degC']=float(abs(numerical-analytic).max())
    assert result['analytic_heat_max_error_degC']<.0002
    (ROOT/'results/verification.json').write_text(json.dumps(result,indent=2))
    print(json.dumps(result,indent=2))

if __name__=='__main__': run()
```

附录 E 结果工作簿导出源程序

文件：code/export_results.mjs

```
// Optional XLSX packaging. Numerical reproduction only requires solve.py.
// Requires @oai/artifact-tool 2.8.58+ in the Codex primary Node runtime.
import fs from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { Workbook, SpreadsheetFile } from '@oai/artifact-tool';
const root = process.env.DRYING_SUPPORT_ROOT || path.resolve(path.dirname(fileURLToPath(import.meta.url)),
  '..');
const out = path.join(root, 'results');
const which = Number(process.argv[2]);
const wb = Workbook.create();
async function csv(name) {
  const lines=(await fs.readFile(path.join(out,name),'utf8')).trim().split(/\r?\n/);
  return lines.slice(1).map(l=>l.split(',').map(v=>v==='nan'?null:Number(v)));
}
async function sheet(name,file,surface=false,radius=false) {
  const s=wb.worksheets.add(name), rows=await csv(file);
  const heads=radius ? ['时间/s','半径/cm'] :
    ['时间\到药材中心的距离',...Array.from({length:surface?20:21},(_,i)=>i/10),...(surface?['药材表面']:[])];
  const width=heads.length, height=rows.length+1;
  s.getRangeByIndexes(0,0,1,width).values=[heads];
  for(let i=0;i<rows.length;i+=2000) {
    const block=rows.slice(i,i+2000).map(r=>r.map((v,j)=>j===0||v===null?v:Math.round(v*1e4)/1e4));
    s.getRangeByIndexes(i+1,0,block.length,width).values=block;
  }
  const used=s.getRangeByIndexes(0,0,height,width);
  used.format.font={name:'Droid Sans Fallback',size:10};
  used.format.rowHeight=19;
  used.format.columnWidth=12;
  used.format.verticalAlignment='center';
  s.getRangeByIndexes(0,0,height,1).format.columnWidth=34;
  s.getRangeByIndexes(0,0,1,width).format={fill:'#E8ECF1',font:
{bold:true,color:'#111827'},rowHeight:30,horizontalAlignment:'center'};
  s.getRangeByIndexes(1,1,rows.length,width-1).setNumberFormat('0.0000');
  s.getRangeByIndexes(1,0,rows.length,1).setNumberFormat('0');
  s.freezePanes.freezeRows(1);s.freezePanes.freezeColumns(1);
  s.showGridLines=true;
}
if(which===1||which===2) {
  await sheet('温度`,`result${which}_T.csv`);
  await sheet('水分浓度`,`result${which}_C.csv`);
} else {
  await sheet('Sheet1`,`result${which}_C.csv`,which===4);
  if(which===4) await sheet('表面位置`,`radius_output.csv`,false,true);
}
wb.recalculate();
console.log((await wb.inspect({kind:'table',range:`${which}<=2?'温度':'Sheet1'}`!
A1:D4`,tableMaxRows:4,tableMaxCols:4,maxChars:1000})).ndjson);
console.log((await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#NUM!',options:
{useRegex:true,maxResults:5},maxChars:1000})).ndjson);
await fs.mkdir(path.join(root,'previews'),{recursive:true});
for(let i=0;i<(which<=2||which===4?2:1);i++) {
  const s=wb.worksheets.getItemAt(i);
  const png=await
wb.render({sheetName:s.name,range:which===4&&i===1?'A1:B8':'A1:G8',scale:1.5,format:'png'});
  await fs.writeFile(path.join(root,'previews`,`result${which}_${i}.png`),new Uint8Array(await
png.arrayBuffer()));
}
await (await SpreadsheetFile.exportXlsx(wb)).save(path.join(root,`result${which}.xlsx`));
console.log(`Exported result${which}.xlsx`);
```